

Informe Técnico - Speedoflight Code Fest Ad Astra

Juan Huertas
jhuertass@unal.edu.co
Universidad Nacional de Colombia
Colombia

Daniel Cortes
dcortesce@unal.edu.co
Universidad Nacional de Colombia
Colombia

Alejandro Padilla
apadillaca@unal.edu.co
Universidad Nacional de Colombia
Colombia

Andrés Leguizamón
aleguizamonl@unal.edu.co
Universidad Nacional de Colombia
Colombia

Introducción

Este documento describe la arquitectura e implementación del *pipeline* de recuperación de información desarrollado para el concurso. El sistema se compone de cinco módulos secuenciales:

- **1. Extracción:** ingiere los distintos formatos del corpus, interpreta su estructura y filtra el contenido relevante.
- **2. Particionamiento (*Chunker*):** segmenta los documentos en fragmentos cohesivos y los tokeniza según el *encoder* utilizado.
- **3. Indexación:** convierte los fragmentos en *embeddings* y construye los índices de similitud con FAISS.
- **4. Motor de búsqueda:** orquesta la inferencia, integra los índices FAISS y el grafo de conocimiento para recuperar los fragmentos relevantes ante cada consulta.
- **5. Grafo de conocimiento:** modela relaciones explícitas entre entidades que complementan la similitud vectorial, mejorando la precisión de recuperación.

La Figura 1 resume el flujo de datos entre estos componentes.

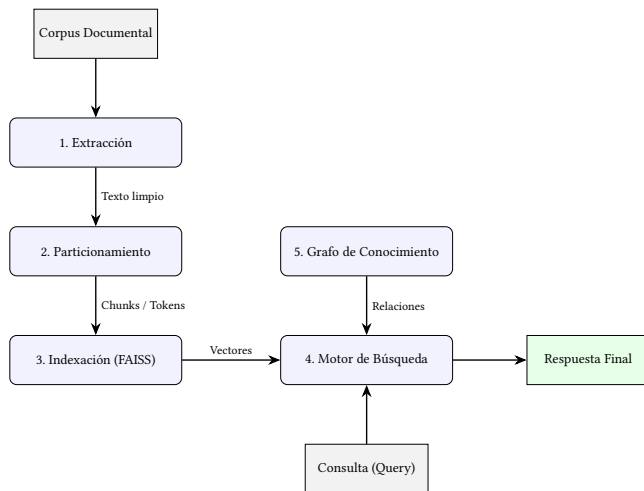


Figure 1: Arquitectura general y flujo de trabajo del *pipeline* de recuperación de información.

Extractor

El primer bloque independiente de código en el cual se decidió trabajar fue la extracción del contenido textual a partir de su formato

original. El equipo organizador del reto suministró una amplia colección de archivos, organizados en carpetas de acuerdo con los tres fenómenos que son de interés para la Fuerza Aeroespacial Colombiana. En la variedad de documentos suministrados se pueden encontrar formatos JSON, PDF, HTML, CSV, XLSX, PNG, JPG y PBF, los cuales en su mayoría correspondían a archivos en formato JSON y PDF.

Clase abstracta Extractor

Puesto que era necesario realizar las dos tareas fundamentales de limpieza y homogeneización de texto para todos los formatos de archivo suministrados, y dado que el procesamiento específico correspondiente a cada uno implica el uso de diferentes librerías y lógicas de extracción, se decidió abordar este primer módulo mediante un paradigma orientado a objetos. De este modo, se definió la clase abstracta *BaseExtractor*, compuesta principalmente por dos métodos abstractos: *tipo_fuente* y *extract_documents*. En el primero se impone a todos los extractores (desarrollados de forma individual y modular) que definan el tipo de archivo que leen, mientras que el segundo establece la forma de extraer la información para cada formato. Así, todas las clases hijas que heredan de la clase abstracta pueden sobrescribir y definir individualmente estos dos métodos y procesos que deben ejecutarse obligatoriamente.

Adicionalmente, sin importar el tamaño, formato o idioma en el cual se encontrase el archivo original, para este primer módulo resulta estrictamente necesario conservar un formato estándar para la salida que contenga la información necesaria y relevante para proceder con el módulo generador de *chunks*. Por esto mismo, se definieron igualmente en la clase abstracta dos métodos adicionales. El primero de ellos, denominado *clean_text*, normaliza espacios, elimina caracteres de control presentes en los múltiples formatos y remueve textos repetitivos o que no son de interés (como encabezados o pies de página recurrentes), además de forzar la puntuación final para mantener la integridad de las oraciones. El segundo método, denominado *process*, se encarga de recibir los documentos en bruto, limpiarlos, detectar su idioma analizando los primeros 2000 caracteres (lo cual permite hacer una confirmación del lenguaje predominante reduciendo la carga computacional) y almacenar todos los resultados en un esquema JSON estricto que incluye el texto extraído, la fuente, el idioma y un identificador único (*doc_id*).

De esta forma, es evidente cómo la decisión de adoptar el paradigma orientado a objetos facilitó todo el procedimiento de diseño, permitiendo trabajar aisladamente en el desarrollo de la lógica y los códigos de procesamiento correspondientes a cada formato de archivo

suministrado, pero conservando estrictamente la homogeneidad en los resultados necesarios para continuar con el siguiente módulo de separación por *chunks*. A continuación, se presenta el formato de datos que se implementó para la salida de todos los archivos procesados.

Formato de salida

A continuación, se presenta de forma condensada y representativa el formato y contenido de los archivos de salida que genera este módulo extractor, los cuales se encuentran en formato .json por facilidad en la integración de módulos:

```
output_schema = {
    "texto": "...",
    "metadata": {
        "total_palabras": 0,
        "atributo_adicional": "...",
        "fuente": "...",
        "tipo_fuente": "...",
        "idioma": "...",
        "doc_id": "...",
        "Fenomeno": "..."
    }
}
```

Librerías utilizadas

El script se apoya en un conjunto robusto de librerías, fundamentales para su operación:

- **pandas**: Fundamental para el manejo de archivos CSV en bloques (*chunks*), lo que evita el desbordamiento de memoria con archivos grandes.
- **BeautifulSoup (bs4)**: Permite analizar la estructura jerárquica del HTML, facilitando la extracción de texto visible y la eliminación de etiquetas irrelevantes como scripts o estilos.
- **pdfplumber**: Extrae de manera precisa el texto y las tablas de los documentos PDF nativos, conservando el diseño estructural de las celdas.
- **pytesseract y PIL**: Habilitan la funcionalidad OCR (Reconocimiento Óptico de Caracteres). PIL mejora el contraste y binariza las imágenes, mientras que pytesseract lee el texto de documentos escaneados.
- **geopandas**: Permite leer archivos de formato PBF y convertirlos a GeoJSON, facilitando la extracción de metadatos geográficos y topológicos.
- **langdetect**: Vital para clasificar el idioma del texto extraído, aplicando lógica de reserva (*fallback*) si el idioma no es soportado.

Fragmentación de texto (Chunking)

Estrategia y justificación

Para la fragmentación del corpus se implementó una estrategia híbrida jerárquico-oracional con solapamiento, construida de forma ascendente (*bottom-up*): el texto se segmenta primero por su estructura en párrafos, cada párrafo se subdivide luego en oraciones completas mediante el tokenizer Punkt de NLTK, y finalmente las

oraciones se agrupan secuencialmente en fragmentos respetando un límite duro de 250 palabras, con solapamiento de oraciones entre fragmentos consecutivos. Esta estrategia combina deliberadamente segmentación estructural, segmentación por oración y superposición semántica, en lugar de aplicar una sola técnica de forma aislada.

Este diseño responde ante todo al requisito de que ningún fragmento contenga oraciones incompletas: en lugar de cortar arbitrariamente por tokens y retroceder hasta el límite de oración más cercano, se construyen los fragmentos agregando oraciones completas una a una y cerrando el fragmento solo cuando la siguiente oración excedería el máximo, lo que garantiza completitud lingüística por construcción. Partir del párrafo como unidad inicial preserva además la intención de segmentación del autor original, evitando mezclar ideas de párrafos no relacionados, y el límite de 250 palabras queda garantizado de forma determinista y sin pasos de recorte posterior, dejando también margen frente al límite típico de 512 tokens de los encoders tipo BERT.

Dado que el corpus mezcla español, inglés y portugués, se utilizó el modelo Punkt de NLTK entrenado específicamente por idioma, que es recibido como metadata del módulo de extracción, sin necesidad de detección que haría mas compleja la implementación, complementado con un diccionario de abreviaturas de dominio (p. ej. “Art.”, “Núm.”) como red de seguridad ante cortes erróneos. El solapamiento entre fragmentos consecutivos mitiga además el riesgo de que una idea quede dividida justo en una frontera. Como limitación, el tamaño de los fragmentos varía según la densidad oracional del texto, y en el caso borde de una oración que por sí sola supera las 250 palabras, esta se conserva completa como excepción justificada, verificada como poco frecuente mediante la función de validación del pipeline.

Formato de Salida y Tokenización por Encoder

Una vez fragmentado cada documento, cada chunk se persiste como un objeto JSON independiente en el archivo `metadata.jsonl` correspondiente, con exactamente los ocho campos exigidos: `doc_id`, `chunk_id`, `fuente`, `formato`, `fenomeno`, `posicion`, `num_tokens` y `texto`. Dado que el proyecto emplea dos encoders operando de forma independiente, el pipeline genera un almacén de metadata separado para cada uno, organizado en subcarpetas `base_vectorial/encoder_<nombre>/metadata.jsonl`. El campo `texto` de cada fragmento es idéntico entre ambos almacenes, ya que la fragmentación es independiente del encoder utilizado; lo único que varía entre ellos es el campo `num_tokens`, recalculado con el tokenizer propio de cada modelo.

La tokenización de cada fragmento no se estima de forma heurística salvo en caso de fallo: para cada encoder definido en la configuración del proyecto, el pipeline carga su tokenizer real mediante `AutoTokenizer.from_pretrained(model_name)` una única vez al inicio del procesamiento y lo reutiliza para todos los documentos de ese encoder. El conteo de tokens de cada chunk se obtiene con `tokenizer.encode(texto, add_special_tokens=False)`, evitando así contaminar la cuenta con tokens especiales (`[CLS]`, `[SEP]`, etc.) que no forman parte del contenido real del fragmento. Si el tokenizer no puede cargarse, por ejemplo por falta de conexión, se recurre a una estimación heurística (`palabras × 1.3`) que permite

continuar el procesamiento sin detener el pipeline, aunque con menor precisión.

Dado que ambos encoders poseen vocabularios y esquemas de subpalabras distintos (WordPiece, BPE o SentencePiece, según el modelo), un mismo fragmento textual puede producir valores de `num_tokens` diferentes en cada almacén de metadata. Este conteo real, calculado por encoder, funciona como verificación empírica de que ningún fragmento supera el límite máximo de entrada del modelo (típicamente 512 tokens): como el límite de fragmentación se fija en 250 palabras de forma independiente del encoder, es este conteo posterior el que confirma, para cada encoder en particular, que dicho límite de palabras se traduce en un número de tokens seguro.

El siguiente fragmento ejemplifica una línea del archivo `metadata.jsonl` de un encoder:

```
{
  "doc_id": "DOC-014",
  "chunk_id": "DOC-014-chunk-003",
  "fuente": "informe_leo_2026.pdf",
  "formato": "pdf",
  "fenomeno": 2,
  "posicion": 3,
  "num_tokens": 187,
  "texto": "..."
```

Junto a cada `metadata.jsonl` se guarda además un archivo lateral `encoder_info.json` con el nombre del modelo y los parámetros de fragmentación usados (`max_words`, `overlap_sentences`) con fines de trazabilidad.

Es importante destacar que este registro se mantiene deliberadamente fuera de `metadata.jsonl`. Esto garantiza que no se altere la correspondencia estricta entre el número de línea y el identificador interno que FAISS asignará a cada vector durante la indexación.

Esta correspondencia línea-a-índice es precisamente la interfaz que consumirá la siguiente etapa del pipeline, encargada de generar los embeddings.

Encoders

Selección de Encoders

Para la selección de los modelos de codificación, se definió un conjunto de criterios que integra los lineamientos del concurso con los requerimientos de implementación del equipo. Tras evaluar las restricciones computacionales y los objetivos de recuperación, el diseño del sistema se fundamentó en la implementación de dos modelos de codificación profunda: BAAI/bge-m3 e `intfloat/multilingual-e5-large`.

A continuación, se detallan los criterios y el análisis que fundamentó esta decisión.

Criterios de Evaluación.

- **Soporte multilingüe unificado:** El corpus exige el procesamiento de documentos en español, inglés y portugués. Dada la restricción computacional (que impide desplegar múltiples modelos especializados) y el requisito de emplear consultas en español para recuperar información en los tres

idiomas, se priorizó un encoder capaz de proyectar las distintas lenguas en un espacio latente compartido. Esto garantiza que la similitud coseno se maximice sin degradación por barreras idiomáticas.

- **Dimensionalidad del vector:** La magnitud del vector de salida (d) afecta linealmente la huella de memoria en el índice vectorial y la latencia en las operaciones de cálculo de distancia. Se requiere un balance entre una mayor dimensionalidad y su impacto en el rendimiento, considerando que aumentar la dimensionalidad no garantiza una ganancia proporcional.
- **Límite de longitud de entrada y particionamiento:** Los encoders presentan límites máximos de tokens (comúnmente 512) que condicionan la estrategia de *chunking*. Las reglas del concurso prohíben fragmentos con oraciones incompletas. Por tanto, un límite restrictivo fuerza a retroceder el corte hasta el final de la última oración completa que no supere el umbral, incrementando el volumen de vectores a indexar y el riesgo de perder dependencias contextuales a largo plazo.
- **Rendimiento empírico en recuperación:** La selección está supeditada al desempeño objetivo en *benchmarks* públicos (MTEB, BEIR), priorizando las métricas en tareas de recuperación densa asimétrica frente a evaluaciones genéricas de clasificación o similitud de pares.
- **Eficiencia computacional y licenciamiento:** Adaptabilidad a los recursos limitados de inferencia de la plataforma destino. Asimismo, los modelos deben cumplir con el licenciamiento abierto (Apache 2.0, MIT, CC BY) exigido por el concurso.
- **Versatilidad representacional e hibridación :** Capacidad estructural del modelo para operar a diferentes niveles de granularidad (oración, párrafo, documento largo) y extraer representaciones densas y dispersas simultáneamente en un único pase de inferencia, optimizando el costo computacional.
- **Alineación para recuperación asimétrica:** Tolerancia del modelo para proyectar secuencias de longitudes dispares (consultas cortas frente a fragmentos extensos) de manera precisa, mitigando el colapso común en encoders no optimizados para la búsqueda.

Análisis Técnico de los Modelos.

BAAI/bge-m3[2]. El modelo bge-m3 (*Multi-Linguality, Multi-Granularity, Multi-Functionality*) se integra como una solución orientada a la versatilidad operativa y la mitigación de las restricciones de contexto:

- **Extensión de contexto:** Su principal ventaja arquitectónica es el soporte para una entrada de hasta 8192 tokens. Esto relaja drásticamente la dificultad de aplicar el *chunking* bajo el estricto requisito de completitud de oraciones. Permite indexar secciones documentales extensas sin fragmentar su cohesión semántica y contiene el crecimiento exponencial del índice FAISS.

- **Versatilidad Multifuncional:** Genera, en una única pasada de inferencia, tres representaciones complementarias: vectores densos ($d = 1024$), vectores dispersos (ponderación léxica aprendida, análoga a BM25 pero entrenada de forma end-to-end) y representaciones multivectoriales estilo ColBERT (a nivel de token, útiles para *re-ranking* fino). Esto permite construir un *pipeline* híbrido (denso + léxico + *re-ranking*) sin el costo de servir tres modelos independientes, reduciendo tanto la latencia de inferencia como la huella de memoria en producción.
- **Licenciamiento y cobertura:** Presenta entrenamiento nativo en más de 100 idiomas y opera bajo una licencia permissiva (MIT), cumpliendo los requerimientos del reto.

`intfloat/multilingual-e5-large[1]`. Este modelo representa un estándar empírico robusto para la alineación semántica interlingüe en escenarios de alta exigencia de precisión:

- **Entrenamiento asimétrico por diseño:** Calibrado mediante aprendizaje contrastivo (InfoNCE) con prefijos de instrucción (`query:` y `passage:`). Esta arquitectura induce una geometría que empareja directamente consultas con documentos, optimizando el rendimiento asimétrico.
- **Consistencia en Benchmarks:** Exhibe un histórico de estado del arte en la clasificación MTEB para recuperación multilingüe. Sus representaciones ($d = 1024$) proporcionan un alto poder discriminativo.
- **Compromiso estructural frente al particionamiento:** Posee una limitación estricta a una ventana de 512 tokens[cite: 2]. Para evitar cortes de oraciones, obliga a adoptar una estrategia de *chunking* conservadora que reduce la longitud efectiva del texto indexado. Aunque esto incrementa la demanda de almacenamiento, su precisión en fragmentos cortos es sobresaliente y facilita el cumplimiento del requisito de salida del concurso (fragmentos recuperados con un límite de 250 palabras).

Síntesis de la Decisión. La selección paralela de `bge-m3` e `intfloat/multilingual-e5-large` establece un marco para abordar el *trade-off* central del sistema. Mientras `multilingual-e5-large` maximiza el rendimiento semántico puro en particiones cortas explotando su preentrenamiento asimétrico, `bge-m3` capitaliza la eficiencia semántica a nivel de documento gracias a su ventana extendida (8192 tokens) que sorteas las limitaciones impuestas por los cortes oracionales. Ambos modelos interactúan para ofrecer un sistema robusto, eficiente y alineado con los requerimientos de la competencia. La elección de ambos modelos permite realizar un *embedding* robusto a varios lenguajes, ventajas de granularidad y limitaciones de extensión sin necesidad de entrenar varios modelos especializados.

Indexación

Implementación Computacional del Indexador

El módulo de indexación vectorial fue desarrollado para automatizar la ingesta de metadatos, la generación de *embeddings* y la construcción del índice FAISS. A continuación, se describen los componentes operativos del sistema y se expone la justificación detrás de la elección de la estructura de índice final.

Mecanismos de Ejecución y Gestión de Hardware. El código implementa rutinas para la administración de los recursos físicos durante la inferencia:

- **Detección de Aceleradores:** Mediante la función `get_optimal_device`, el sistema identifica y asigna el hardware disponible, soportando arquitecturas CUDA/ROCm, DirectML (para GPUs AMD en entornos Windows) y CPU.
- **Gestión de Inferencia en DirectML:** Se emplea un gestor de contexto (`_SafeInferenceMode`) que sobrescribe `torch.inference_mode()` y utiliza `torch.no_grad()`. Este parche soluciona una incompatibilidad nativa de PyTorch al procesar tensores con aceleración DirectML.
- **Precisión FP16:** La instanciación del modelo *Transformer* aplica precisión de media coma flotante (FP16) cuando se detecta soporte por parte de la GPU.

Procesamiento de Datos y Control de Memoria. El flujo de vectorización opera bajo directrices de entrada/salida (I/O) y recolección de memoria controladas:

- **Parseo de Metadatos:** La extracción de textos desde los archivos `metadata.jsonl` se ejecuta utilizando la biblioteca `orjson`.
- **Vectorización por Lotes y Limpieza:** La inferencia se realiza en lotes (`batch_size=128`). Al finalizar cada bloque, el sistema invoca `torch.cuda.empty_cache()` y `gc.wallobreakcollect()` para liberar explícitamente la memoria VRAM y RAM.
- **Normalización L_2 :** Antes de la inserción en la base de datos vectorial, el bloque de *embeddings* final es sometido a una normalización L_2 global a través de `faiss.normalize_L2`. Al utilizar la métrica de producto interno, esta normalización produce un cálculo equivalente a la similitud coseno. Este mecanismo es robusto en caso de normalizaciones previas realizadas por error: dado que la normalización L_2 es una operación idempotente, renormalizar un vector que ya está normalizado no altera su valor, por lo que aplicar el paso dos veces no introduce ningún efecto adverso.

Justificación de la Estructura de Índice (IndexFlatIP). El orquestador implementa un patrón de diseño *Strategy* (`FaissIndexStrategy`) que permitió construir y evaluar de forma concurrente tres enfoques de indexación durante la fase de desarrollo: `FlatIndexStrategy` (búsqueda exacta), `IVFFlatIndexStrategy` (cuantificación invertida) y `HNSWIndexStrategy` (grafos de mundo pequeño).

Tras la evaluación operativa, se tomó la decisión de descartar las arquitecturas aproximadas (IVF y HNSW) y decantarnos exclusivamente por el índice plano (`IndexFlatIP`). Esta elección de diseño se justifica por los siguientes dos motivos:

- (1) **Magnitud del Corpus:** El volumen total de fragmentos de texto vectorizados tras el particionamiento no alcanzó la escala crítica (del orden de millones de vectores) que haría necesario el uso de algoritmos de Búsqueda Aproximada del Vecino Más Cercano (ANN). Para el tamaño de este corpus, la latencia de una búsqueda exhaustiva (fuerza bruta) en un índice plano es completamente eficiente y no representa un cuello de botella.

- (2) **Determinismo para Pruebas y Evaluación:** A diferencia de los índices IVF y HNSW, que introducen estocasticidad y devuelven resultados aproximados basados en agrupamiento o topologías de grafos, IndexFlatIP garantiza un comportamiento estrictamente determinista. Devolver el conjunto exacto y preciso de vecinos más cercanos es un requerimiento crucial para la fase de *testing*, ya que asegura que las métricas de evaluación sean reproducibles y confiables ante consultas idénticas.

Motor de Búsqueda

Justificación del Diseño

El motor de búsqueda fue diseñado bajo una arquitectura de recuperación puramente vectorial y determinista. La selección de una estrategia *Multi-Encoder* responde a la necesidad de mitigar el sesgo semántico que puede generarse al usar un único modelo.

Pasos de Ejecución

- (1) **Preprocesamiento y Corrección Ortográfica:** La consulta ingresada por el usuario es sometida a una normalización Unicode (NFC) y limpieza de caracteres de control. Para evitar la alteración de siglas y acrónimos técnicos (identificados como cadenas continuas en mayúsculas), el sistema omite selectivamente la corrección ortográfica. Para el resto de las consultas, se utiliza la herramienta SymSpell con una distancia máxima de edición $d = 2$, evaluada contra un diccionario global construido a partir del corpus indexado. Durante la creación de este diccionario, el vocabulario es normalizado a minúsculas y filtrado mediante expresiones regulares para preservar de manera estricta los caracteres diacríticos multilingües.
- (2) **Recuperación Vectorial Multi-Encoder:** La consulta normalizada se vectoriza en paralelo utilizando dos modelos: BAAI/bge-m3 e `intfloat/multilingual-e5-large` (aplicando el prefijo requerido por este último). Los vectores resultantes son normalizados bajo la norma L_2 y consultados contra índices FAISS. Durante esta fase se aplican filtros heurísticos para descartar fragmentos con ruido de codificación (por ejemplo, caracteres no correspondientes a los idiomas del corpus).
- (3) **Fusión de Rankings (RRF) y Deduplicación:** Las listas de candidatos (*Top-50*) de ambos índices se consolidan utilizando el algoritmo Reciprocal Rank Fusion (RRF) con una constante de suavizado $k_0 = 60$:

$$s_{RRF}(c) = \sum_{j=1}^2 \frac{1}{60 + r_j(c)} \quad (1)$$

Para evitar redundancias en la respuesta final, se calcula un identificador criptográfico (hash MD5) sobre el texto normalizado de cada fragmento. Los fragmentos con contenido textual idéntico, provenientes de distintas fuentes o metadatos, se fusionan conservando la puntuación RRF acumulada.

- (4) **Alineación de Resultados:** El sistema selecciona los mejores 10 fragmentos y asegura, la extracción secuencial de exactamente 3 identificadores únicos de documento (`doc_id`).

Grafo de Conocimiento

Justificación del Diseño

El grafo de conocimiento se implementó como una estructura complementaria para modelar y trazar las relaciones explícitas entre las entidades mencionadas en los documentos. Su propósito es habilitar análisis relacionales y visualizaciones estructuradas.

Pasos de Creación

- (1) **Consolidación de Metadatos:** El proceso inicia iterando sobre los directorios de los encoders para leer los archivos `metadata.jsonl`. Se realiza una validación cruzada y deduplicación utilizando el `chunk_id` como clave principal, consolidando un único conjunto de fragmentos de texto válidos.
- (2) **Procesamiento de Lenguaje Natural (NLP):** Cada fragmento de texto es analizado mediante la librería `langdetect` para identificar su idioma predominante. Con base en esta detección, se carga el modelo acústico y sintáctico correspondiente de la librería `SpaCy` (`es\core\news\_sm`, `en\core_web\_sm` o `pt\core_news\_sm`).
- (3) **Reconocimiento de Entidades y Extracción de Relaciones:** El texto se divide en oraciones. Dentro de cada oración, se extraen las entidades nombradas (nodos). Si existen al menos dos entidades en una misma oración, se extrae el lema de los verbos principales y auxiliares que las conectan para definir la relación semántica (arista). En caso de no detectar un verbo de conexión claro, se asigna la relación estandarizada `relacionado_con`.
- (4) **Construcción Estructurada y Trazabilidad:** Utilizando la librería `NetworkX`, se instancia un grafo dirigido (`nx.DiGraph`). Los nodos almacenan el texto de la entidad y su tipo de entidad nombrada (NER, por sus siglas en inglés, identificando categorías como organizaciones, locaciones o personas). Las aristas (relaciones) almacenan atributos de trazabilidad: `doc_id`, `chunk_id` y `fenomeno`. Si una misma relación entre dos entidades ocurre en múltiples fragmentos, los identificadores se agregan y separan por comas.
- (5) **Exportación:** El modelo estructurado se exporta de manera nativa al formato estándar `GraphML` (`grafo.graphml`), asegurando que la topología y los metadatos puedan ser ingeridos sin alteraciones por bases de datos orientadas a grafos.

References

- [1] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-embedding: multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*. Lun-Wei Ku, Andre Martins, and Vivek Srikumar, (Eds.) Association for Computational Linguistics, Bangkok, Thailand, (Aug. 2024), 2318–2335. doi:10.18653/v1/2024.findings-acl.137.
- [2] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024. Multilingual e5 text embeddings: a technical report. (2024). <https://arxiv.org/abs/2402.05672> arXiv: 2402.05672 [cs.CL].